

VAULT

Multi-Tenant AI Knowledge Platform

Backend Developer Task Plan / PM-Style Work Package

Version 1.0 • September 2026

Project type	Portfolio-grade production-style backend project
Primary stack	Python, FastAPI, SQLAlchemy/SQLAlchemy, PostgreSQL, Alembic
Later stack	Celery, Redis, Qdrant, Ollama, Prometheus, Docker Compose
Security invariant	A user must never access another organization’s private data

1. How to Read This Work Package

This document is written in the style of a project manager / product owner handoff to a backend developer. Each ticket describes the business goal, the developer work expected, dependencies, and acceptance criteria. The developer is expected to clarify unclear requirements before implementation rather than silently inventing product behavior.

- Epic / Sprint: groups related business outcomes into a time-boxed development increment.
- Ticket: a piece of work small enough to implement, review, test, and mark Done.
- User story / goal: explains why the work exists, not just what file to create.
- Developer tasks: implementation activities expected from the engineer.
- Acceptance criteria: observable conditions that must be true before the ticket is accepted.
- Dependencies: work that must exist first or another component the ticket relies on.

2. Product Brief

Vault is a backend service where teams upload documents and ask natural-language questions against their own knowledge base. Answers must be grounded in the organization's documents and return source references. The engineering challenge is the production system around RAG: tenant isolation, reliable asynchronous ingestion, stable API contracts, observability, testing, and deployment.

3. Global Engineering Rules

- Tenant isolation is enforced in backend/query logic; UI hiding is never treated as security.
- Raw passwords are never stored or logged.
- Database schema changes are versioned with Alembic migrations.
- Large/expensive document ingestion is asynchronous and must not block the upload request.
- A failed document or model call must produce a readable failure state instead of silently killing a worker.
- RAG answers must be sourced; unsourced answers are not accepted as a successful response.
- Tests must not call real embedding/LLM models when mocks are sufficient.
- Configuration and secrets come from environment/configuration, not hardcoded source code.
- Each sprint is complete only when acceptance criteria and required tests pass.

4. Definition of Done (All Tickets)

- Code is implemented and locally runnable.
- Relevant happy-path and failure-path behavior has been tested.
- No secrets or sensitive data are exposed in source, responses, or logs.
- Database changes include a reviewed migration when applicable.
- The API response contract is stable and documented by FastAPI/OpenAPI schemas.
- Existing tests remain green.
- The developer can explain the design decision and trade-offs during review.

Sprint 1 — Foundations: Authentication & Multi-Tenancy

Sprint objective: Establish identity, authentication, and tenant isolation before any document or AI features are introduced.

Ticket	Title	Priority	Depends on	Status
VLT-101	Bootstrap FastAPI Application	High	—	Done
VLT-102	Configure Application Settings and PostgreSQL Connection	High	VLT-101	Done
VLT-103	Create Organization and User Data Models	Critical	VLT-102	Done
VLT-104	Add Alembic Schema Migrations	High	VLT-103	Done
VLT-105	Define Authentication API Schemas	High	VLT-103	Done
VLT-106	Implement Password Hashing and Verification	Critical	VLT-105	Done
VLT-107	Implement JWT Creation and Decoding	Critical	VLT-106	Done
VLT-108	Implement Current User Dependency	Critical	VLT-107	Done
VLT-109	Implement Organization Scope Dependency	Critical	VLT-108	Done
VLT-110	Implement User Service / Reusable User Lookups	Medium	VLT-103	Done
VLT-111	Implement Signup Endpoint	Critical	VLT-105,VLT-106,VLT-110	Done
VLT-112	Implement Login Endpoint	Critical	VLT-107,VLT-110,VLT-111	Next
VLT-113	Add Current User Protected Endpoint	High	VLT-108,VLT-112	To Do
VLT-114	Add Tenant Isolation Demonstration Endpoint	Critical	VLT-109,VLT-112	To Do

Ticket	Title	Priority	Depends on	Status
VLT-115	Automate Authentication and Tenant Boundary Tests	Critical	VLT-111,VLT-112,VLT-114	To Do

Sprint 1 Detailed Tickets

VLT-101 — Bootstrap FastAPI Application

Sprint	1	Priority	High
Depends on	—	Status	Done

User story / goal: As a developer, I need a clean FastAPI application entry point so the backend can start predictably and expose API routes.

Developer tasks

- Create the Python project and virtual environment.
- Create app/main.py and initialize FastAPI.
- Add basic application metadata and a simple health/root route.
- Create .gitignore and keep .env and .venv out of version control.

Acceptance criteria

- Application starts with Uvicorn without errors.
- GET /docs loads OpenAPI documentation.
- No database/authentication logic is embedded directly in main.py.

VLT-102 — Configure Application Settings and PostgreSQL Connection

Sprint	1	Priority	High
Depends on	VLT-101	Status	Done

User story / goal: As a backend service, Vault needs environment-based configuration and a working PostgreSQL connection.

Developer tasks

- Create app/core/config.py using Pydantic Settings.
- Create .env values for database URL and JWT settings.
- Create app/db/session.py and database session dependency.
- Verify connection with a simple SELECT 1.

Acceptance criteria

- Configuration loads from .env.
- Python connects to the vault PostgreSQL database successfully.
- Database sessions are closed after use.
- Secrets are not hardcoded in source code.

VLT-103 — Create Organization and User Data Models

Sprint	1	Priority	Critical
Depends on	VLT-102	Status	Done

User story / goal: As Vault, I need users to belong to exactly one organization so tenant ownership can be enforced.

Developer tasks

- Create Organization table model.
- Create User table model.
- Add unique user email constraint/index.
- Store hashed_password, never a raw password.
- Add users.org_id foreign key to organizations.id.

Acceptance criteria

- Organization and User models load successfully.
- A User requires an organization ID.
- Email uniqueness is enforced at database level.
- User model contains hashed_password and no raw password column.

VLT-104 — Add Alembic Schema Migrations

Sprint	1	Priority	High
Depends on	VLT-103	Status	Done

User story / goal: As an engineering team, we need repeatable version-controlled database schema changes.

Developer tasks

- Initialize Alembic.
- Connect Alembic to SQLAlchemy metadata and application database URL.
- Generate initial organizations/users migration.
- Review autogenerated migration before applying.
- Apply migration to PostgreSQL.

Acceptance criteria

- alembic upgrade head succeeds.
- organizations, users, and alembic_version tables exist.
- users.org_id references organizations.id.

VLT-105 — Define Authentication API Schemas

Sprint	1	Priority	High
Depends on	VLT-103	Status	Done

User story / goal: As an API consumer, I need stable signup, login, and token request/response contracts.

Developer tasks

- Create UserSignUp schema.
- Create UserLogin schema.
- Create TokenResponse schema.
- Validate emails and basic password requirements.

Acceptance criteria

- Invalid email input is rejected by schema validation.
- Signup input does not expose backend-controlled fields such as id or hashed_password.
- TokenResponse contains access_token and token_type only.

VLT-106 — Implement Password Hashing and Verification

Sprint	1	Priority	Critical
Depends on	VLT-105	Status	Done

User story / goal: As a user, my password must be stored securely and verified without storing plaintext credentials.

Developer tasks

- Implement hash_password().
- Implement verify_password().
- Use a trusted password hashing library.

Acceptance criteria

- Signup stores a password hash rather than plaintext.
- Correct password verifies True.
- Incorrect password verifies False.

VLT-107 — Implement JWT Creation and Decoding

Sprint	1	Priority	Critical
Depends on	VLT-106	Status	Done

User story / goal: As an authenticated user, I need a signed access token that identifies both me and my organization.

Developer tasks

- Implement create_access_token(user_id, org_id).
- Include sub, org_id, and exp claims.
- Implement decode_token().
- Reject invalid/expired tokens.

Acceptance criteria

- Created token contains user identity and org_id.
- Modified or expired tokens are rejected.
- JWT secret and algorithm come from configuration.

VLT-108 — Implement Current User Dependency

Sprint	1	Priority	Critical
Depends on	VLT-107	Status	Done

User story / goal: As a protected API route, I need a reusable way to identify the authenticated database user from a bearer token.

Developer tasks

- Extract bearer token using FastAPI security dependency.
- Decode token.
- Read sub claim.
- Load the user from PostgreSQL.
- Return 401 when token is invalid or user no longer exists.

Acceptance criteria

- Valid token returns the real User object.
- Missing/invalid token returns 401.
- A valid token for a deleted/nonexistent user returns 401.

VLT-109 — Implement Organization Scope Dependency

Sprint	1	Priority	Critical
Depends on	VLT-108	Status	Done

User story / goal: As Vault, I must prevent authenticated users from accessing another organization's resources.

Developer tasks

- Implement require_org_scope().
- Compare current_user.org_id with requested organization ID.
- Return 403 on mismatch.

Acceptance criteria

- Same-organization request is allowed.
- Cross-organization request returns 403 Forbidden.
- Authorization uses the current database user's organization membership.

VLT-110 — Implement User Service / Reusable User Lookups

Sprint	1	Priority	Medium
Depends on	VLT-103	Status	Done

User story / goal: As a developer, I need reusable user/organization lookup and creation logic so routes stay focused on HTTP behavior.

Developer tasks

- Implement `get_user_by_email()`.
- Implement `get_user_by_id()`.
- Implement `user_exists()`.
- Implement organization lookup/existence helpers as currently needed.
- Implement `create_user()` with password hashing.

Acceptance criteria

- Signup/login can reuse service methods instead of duplicating lookup queries.
- Service does not expose raw password storage.
- Transaction commit boundary remains controlled by the caller for multi-step operations.

VLT-111 — Implement Signup Endpoint

Sprint	1	Priority	Critical
Depends on	VLT-105,VLT-106,VLT-110	Status	Done

User story / goal: As a new customer, I want to create an organization and its first user account.

Developer tasks

- Create POST `/auth/signup`.
- Reject duplicate user email.
- Create organization and first user in one transaction.
- Hash password before persistence.
- Return only safe user fields.

Acceptance criteria

- Valid request returns 201 Created.
- Organization and user rows are created and linked by `org_id`.
- Duplicate email is rejected.
- Database contains a hash, not the submitted password.
- `hashed_password` is not returned.

VLT-112 — Implement Login Endpoint

Sprint	1	Priority	Critical
Depends on	VLT-107,VLT-110,VLT-111	Status	Next

User story / goal: As a registered user, I want to log in with email/password and receive an access token for protected Vault APIs.

Developer tasks

- Create POST /auth/login.
- Find user by email.
- Verify supplied password against stored hash.
- Create JWT using user.id and user.org_id.
- Return TokenResponse.

Acceptance criteria

- Valid credentials return 200 and a bearer access token.
- Unknown email returns 401.
- Wrong password returns 401.
- Decoded token includes sub, org_id, and exp.
- No password/hash is returned.

VLT-113 — Add Current User Protected Endpoint

Sprint	1	Priority	High
Depends on	VLT-108,VLT-112	Status	To Do

User story / goal: As an authenticated user, I want to retrieve my own profile so authentication can be validated end-to-end.

Developer tasks

- Create GET /users/me or equivalent protected endpoint.
- Use Depends(get_current_user).
- Return safe user fields.

Acceptance criteria

- Valid token returns current user data.
- Missing, invalid, or expired token returns 401.
- Password hash is never included.

VLT-114 — Add Tenant Isolation Demonstration Endpoint

Sprint	1	Priority	Critical
Depends on	VLT-109,VLT-112	Status	To Do

User story / goal: As the product owner, I need proof that Org A cannot directly access Org B scoped resources.

Developer tasks

- Create a simple organization-scoped protected resource/endpoint.
- Use require_org_scope().
- Create two organizations and users for verification.

Acceptance criteria

- Org A user accessing Org A resource is allowed.
- Org A user accessing Org B resource by direct ID returns 403, not empty results.

VLT-115 — Automate Authentication and Tenant Boundary Tests

Sprint	1	Priority	Critical
Depends on	VLT-111,VLT-112,VLT-114	Status	To Do

User story / goal: As an engineering team, we need regression tests proving authentication and tenant isolation continue to work.

Developer tasks

- Add signup success/duplicate tests.
- Add login success/wrong password/unknown email tests.
- Add missing/invalid token tests.
- Add Org A → Org B forbidden test.

Acceptance criteria

- Test suite runs locally and passes.
- Tenant isolation test explicitly proves cross-org access returns 403.
- Tests do not depend on manual Postman steps.

Sprint 2 — Asynchronous Document Ingestion

Sprint objective: Accept PDFs, process them reliably in background jobs, and store organization-scoped vectors with observable status.

Ticket	Title	Priority	Depends on	Status
VLT-201	Create Document Metadata Model	Critical	Sprint 1	To Do
VLT-202	Implement PDF Upload Endpoint	Critical	VLT-201	To Do
VLT-203	Add Redis and Celery Background Processing	Critical	VLT-202	To Do
VLT-204	Implement PDF Extraction and Chunking	High	VLT-203	To Do
VLT-205	Implement Embedding Adapter	High	VLT-204	To Do
VLT-206	Store Chunks in Qdrant with Tenant Metadata	Critical	VLT-205	To Do
VLT-207	Implement Document Status Endpoint	High	VLT-201,VLT-203	To Do
VLT-208	Test Ingestion Pipeline and Failure Handling	Critical	VLT-204,VLT-205,VLT-206,VLT-207	To Do

Sprint 2 Detailed Tickets

VLT-201 — Create Document Metadata Model

Sprint	2	Priority	Critical
Depends on	Sprint 1	Status	To Do

User story / goal: As Vault, I need persistent document ownership and ingestion state before processing files.

Developer tasks

- Create Document model with `org_id` ownership.
- Add filename/storage reference/status/error fields.
- Define pending, processing, ready, failed states.
- Generate and review Alembic migration.

Acceptance criteria

- Document rows are organization-owned.

- New upload can be represented as pending.
- Migration applies successfully.

VLT-202 — Implement PDF Upload Endpoint

Sprint	2	Priority	Critical
Depends on	VLT-201	Status	To Do

User story / goal: As an organization user, I want to upload a PDF without waiting for full ingestion to finish.

Developer tasks

- Create multipart PDF upload endpoint.
- Validate file type/basic size constraints.
- Store file or persistent file reference.
- Create Document row as pending.
- Return document ID/status immediately.

Acceptance criteria

- Valid PDF returns quickly with pending status.
- Uploaded document is tied to current user org_id.
- Cross-org ownership cannot be supplied by client.

VLT-203 — Add Redis and Celery Background Processing

Sprint	2	Priority	Critical
Depends on	VLT-202	Status	To Do

User story / goal: As the API, I need expensive ingestion to run outside the HTTP request lifecycle.

Developer tasks

- Configure Redis broker.
- Configure Celery worker.
- Enqueue ingestion task after upload.
- Pass document identifier, not large raw objects, to worker.

Acceptance criteria

- Upload request returns before ingestion completes.
- Worker receives queued task.
- API remains responsive while task runs.

VLT-204 — Implement PDF Extraction and Chunking

Sprint	2	Priority	High
Depends on	VLT-203	Status	To Do

User story / goal: As the ingestion worker, I need to convert valid PDFs into retrievable text chunks.

Developer tasks

- Extract PDF text.
- Handle empty/corrupt PDFs.
- Implement deterministic chunking strategy.
- Preserve document/chunk metadata.

Acceptance criteria

- Valid PDF produces chunks.
- Corrupt PDF produces controlled failure.
- One bad PDF does not crash the worker process.

VLT-205 — Implement Embedding Adapter

Sprint	2	Priority	High
Depends on	VLT-204	Status	To Do

User story / goal: As the ingestion pipeline, I need embeddings through a replaceable interface rather than hardcoding one model implementation.

Developer tasks

- Define embedding interface/adaptor.
- Implement local embedding provider.
- Add timeout/error handling.

Acceptance criteria

- Embedding implementation can be swapped without rewriting ingestion flow.
- Embedding failure moves document to failed with readable error.

VLT-206 — Store Chunks in Qdrant with Tenant Metadata

Sprint	2	Priority	Critical
Depends on	VLT-205	Status	To Do

User story / goal: As Vault, I need vectorized chunks stored with organization metadata so later retrieval can enforce tenancy.

Developer tasks

- Configure Qdrant collection.
- Store chunk vectors and metadata.
- Include org_id and document_id on every vector record.

Acceptance criteria

- Chunks exist in Qdrant after successful ingestion.
- Every stored chunk includes correct org_id.
- Document status becomes ready only after vector storage succeeds.

VLT-207 — Implement Document Status Endpoint

Sprint	2	Priority	High
Depends on	VLT-201,VLT-203	Status	To Do

User story / goal: As a user, I want to check whether my document is pending, processing, ready, or failed.

Developer tasks

- Create GET document status endpoint.
- Enforce current-user organization ownership.
- Return readable failure message when failed.

Acceptance criteria

- Owner can query status.
- Other org receives 403.
- Corrupt file results in failed status and readable error.

VLT-208 — Test Ingestion Pipeline and Failure Handling

Sprint	2	Priority	Critical
Depends on	VLT-204,VLT-205,VLT-206,VLT-207	Status	To Do

User story / goal: As engineering, we need proof that valid and invalid ingestion paths are reliable.

Developer tasks

- Test pending → ready flow with a real sample PDF.
- Test corrupt PDF pending → failed.
- Mock embedding provider in automated tests.
- Verify Qdrant metadata org_id.

Acceptance criteria

- Happy path passes.
- Corrupt PDF fails gracefully.
- Tests do not call a real embedding model.

Sprint 3 — Tenant-Safe RAG Querying

Sprint objective: Retrieve only the requester's organization data and return grounded answers with sources through a swappable LLM layer.

Ticket	Title	Priority	Depends on	Status
VLT-301	Define Query API Contract	High	Sprint 2	To Do
VLT-302	Implement Tenant-Filtered Vector Retrieval	Critical	VLT-301,VLT-206	To Do
VLT-303	Create Swappable LLM Provider Interface	High	VLT-301	To Do
VLT-304	Implement Grounded RAG Query Flow	Critical	VLT-302,VLT-303	To Do
VLT-305	Automate Cross-Tenant RAG Isolation Test	Critical	VLT-304	To Do

Sprint 3 Detailed Tickets

VLT-301 — Define Query API Contract

Sprint	3	Priority	High
Depends on	Sprint 2	Status	To Do

User story / goal: As an API consumer, I need a stable request/response contract for asking questions and receiving sourced answers.

Developer tasks

- Create query request schema.
- Create sourced answer response schema.
- Define source fields such as document ID/name and optional chunk reference.

Acceptance criteria

- OpenAPI documents request and response.
- Response contract always contains sources for successful answers.

VLT-302 — Implement Tenant-Filtered Vector Retrieval

Sprint	3	Priority	Critical
Depends on	VLT-301,VLT-206	Status	To Do

User story / goal: As Vault, I must retrieve only chunks belonging to the requesting organization.

Developer tasks

- Embed incoming question.
- Search Qdrant by similarity.
- Apply mandatory org_id filter derived from authenticated user.
- Select top relevant chunks.

Acceptance criteria

- Query never searches another org's vectors.
- org_id is derived from authenticated context, not client-controlled input.

VLT-303 — Create Swappable LLM Provider Interface

Sprint	3	Priority	High
Depends on	VLT-301	Status	To Do

User story / goal: As engineering, I want Ollama now but the ability to switch to a hosted LLM through configuration later.

Developer tasks

- Define LLM provider interface.
- Implement Ollama provider.
- Move provider selection/settings to configuration.

Acceptance criteria

- Query flow depends on interface, not Ollama-specific code.
- Changing provider does not require rewriting route/business flow.

VLT-304 — Implement Grounded RAG Query Flow

Sprint	3	Priority	Critical
Depends on	VLT-302,VLT-303	Status	To Do

User story / goal: As a user, I want an answer based on my organization's documents with the supporting sources.

Developer tasks

- Build prompt from question + retrieved chunks.
- Call LLM provider.
- Return answer and source documents.
- Handle insufficient context without inventing an unsourced answer.

Acceptance criteria

- Successful answer contains source references.
- No retrieved evidence means system does not present an unsupported sourced answer.
- Query latency/error handling is reasonable.

VLT-305 — Automate Cross-Tenant RAG Isolation Test

Sprint	3	Priority	Critical
Depends on	VLT-304	Status	To Do

User story / goal: As the product owner, I need an automated proof that similar questions cannot leak another organization's content.

Developer tasks

- Seed Org A and Org B with different documents.
- Query as Org A with terms similar to Org B content.
- Assert Org B content/sources are never returned.

Acceptance criteria

- Cross-org retrieval leakage test passes repeatedly.
- Test is automated, not only a manual spot-check.

Sprint 4 — Observability & Automated Quality

Sprint objective: Make the system measurable, test tenant boundaries thoroughly, and make ingestion/query failures diagnosable.

Ticket	Title	Priority	Depends on	Status
VLT-401	Expose Prometheus Metrics Endpoint	High	Sprints 2-3	To Do
VLT-402	Instrument Ingestion Metrics	High	VLT-401,VLT-208	To Do
VLT-403	Instrument Query Latency	High	VLT-401,VLT-304	To Do
VLT-404	Complete Tenancy Boundary Test Suite	Critical	VLT-115,VLT-305	To Do
VLT-405	Complete Ingestion Tests with Mocked Embeddings	Critical	VLT-208	To Do

Sprint 4 Detailed Tickets

VLT-401 — Expose Prometheus Metrics Endpoint

Sprint	4	Priority	High
Depends on	Sprints 2-3	Status	To Do

User story / goal: As operations/engineering, I need scrapeable metrics to observe Vault in production.

Developer tasks

- Add Prometheus client integration.
- Expose /metrics.
- Ensure endpoint format is Prometheus compatible.

Acceptance criteria

- Prometheus can scrape /metrics.
- Metrics endpoint does not expose secrets.

VLT-402 — Instrument Ingestion Metrics

Sprint	4	Priority	High
Depends on	VLT-401,VLT-208	Status	To Do

User story / goal: As engineering, I want visibility into ingestion duration and failures.

Developer tasks

- Measure ingestion job duration.

- Add failed-ingestion counter.
- Use meaningful metric names/labels without high-cardinality IDs.

Acceptance criteria

- Valid and failed jobs change expected metrics.
- Metrics can distinguish failures without leaking user/document content.

VLT-403 — Instrument Query Latency

Sprint	4	Priority	High
Depends on	VLT-401,VLT-304	Status	To Do

User story / goal: As engineering, I want query latency metrics so slow RAG behavior is diagnosable.

Developer tasks

- Measure query path latency.
- Expose histogram/summary suitable for Prometheus.

Acceptance criteria

- Each query updates latency metric.
- Metric does not use user/question text as labels.

VLT-404 — Complete Tenancy Boundary Test Suite

Sprint	4	Priority	Critical
Depends on	VLT-115,VLT-305	Status	To Do

User story / goal: As security/review, I need comprehensive automated coverage of tenant boundaries across current resources.

Developer tasks

- Cover document access, status, query, and direct resource ID attempts.
- Cover missing/invalid authentication.
- Review every protected route for shared dependency usage.

Acceptance criteria

- Org A cannot retrieve Org B documents through any tested endpoint.
- Regression suite fails if tenancy enforcement is removed.

VLT-405 — Complete Ingestion Tests with Mocked Embeddings

Sprint	4	Priority	Critical
Depends on	VLT-208	Status	To Do

User story / goal: As engineering, I need deterministic ingestion tests that do not depend on external/local models.

Developer tasks

- Mock embedding generation.
- Test timeout/failure branches.
- Test status transitions and error persistence.

Acceptance criteria

- Tests run without loading a real embedding model.
- Success and failure state transitions are deterministic.

Sprint 5 — Deployment & Portfolio Polish

Sprint objective: Create a reproducible containerized stack, deploy it live, and document/demo the system professionally.

Ticket	Title	Priority	Depends on	Status
VLT-501	Containerize Full Local Stack	Critical	Sprints 1-4	To Do
VLT-502	Deploy Vault to a Live Environment	Critical	VLT-501	To Do
VLT-503	Write Portfolio-Grade README and Architecture Documentation	High	VLT-502	To Do
VLT-504	Record Short End-to-End Demo	Medium	VLT-502,VLT-503	To Do

Sprint 5 Detailed Tickets

VLT-501 — Containerize Full Local Stack

Sprint	5	Priority	Critical
Depends on	Sprints 1-4	Status	To Do

User story / goal: As a developer/reviewer, I want one reproducible command to run the complete Vault stack locally.

Developer tasks

- Create Dockerfile for API.
- Create Docker Compose services for API, PostgreSQL, Redis, Celery worker, Qdrant, and required local AI service/config.
- Add volumes, health checks, environment handling, and service networking.
- Run Alembic migrations in containerized workflow.

Acceptance criteria

- Fresh machine can start the supported local stack using documented commands.
- Services communicate by container service names.
- Persistent services use volumes where appropriate.

VLT-502 — Deploy Vault to a Live Environment

Sprint	5	Priority	Critical
Depends on	VLT-501	Status	To Do

User story / goal: As a portfolio reviewer, I want a real running Vault API that demonstrates production deployment skills.

Developer tasks

- Choose supported hosting platform.
- Configure production environment variables/secrets.
- Provision required managed/self-hosted services within platform constraints.
- Apply database migrations.
- Verify health and key API flows.

Acceptance criteria

- Live API URL is reachable.
- Signup/login/upload/query demonstration works in supported deployment configuration.
- Secrets are managed outside source code.

VLT-503 — Write Portfolio-Grade README and Architecture Documentation

Sprint	5	Priority	High
Depends on	VLT-502	Status	To Do

User story / goal: As a reviewer, I want to quickly understand the problem, architecture, setup, security design, and limitations.

Developer tasks

- Write project overview.
- Add architecture diagram.
- Document local setup and migration commands.
- Document API flow and tenant isolation approach.
- List honest limitations and next steps.

Acceptance criteria

- New reviewer can understand and run the project from README.
- Architecture and tenancy decisions are explicitly documented.

VLT-504 — Record Short End-to-End Demo

Sprint	5	Priority	Medium
Depends on	VLT-502,VLT-503	Status	To Do

User story / goal: As a recruiter/reviewer, I want a concise demonstration of the product working end-to-end.

Developer tasks

- Record approximately two-minute demo.
- Show authentication, PDF upload, processing status, question, sourced answer.
- Keep demo focused on product value and engineering behavior.

Acceptance criteria

- Demo clearly shows upload → processing → sourced answer.

- No credentials/secrets are visible in recording.

5. Sprint Exit Criteria

Sprint 1

- Two organizations and users can be created.
- Users can sign up and log in.
- JWT contains sub and org_id.
- Protected endpoints reject invalid authentication.
- Org A direct access to Org B returns 403.
- Automated tenant-boundary test passes.

Sprint 2

- Valid PDF moves pending → ready.
- Corrupt PDF moves to failed with readable error.
- Worker remains healthy after bad input.
- Qdrant chunks include correct org_id metadata.
- Document status is queryable and tenant-scoped.

Sprint 3

- Question returns grounded answer plus sources.
- Qdrant retrieval always filters by requester org_id.
- Org A automated query cannot surface Org B content.
- LLM provider can be swapped through configuration/adapter.

Sprint 4

- /metrics is Prometheus-scrapeable.
- Ingestion duration, query latency, and failed ingestion count are exposed.
- Core tenancy and ingestion regression tests pass.
- Embedding calls are mocked in tests.

Sprint 5

- Full stack runs through Docker Compose locally.
- A live deployment is available.
- README explains architecture, setup, limitations, and next steps.
- Short demo shows upload and sourced Q&A.

6. Recommended Daily Developer Workflow

1. Read the ticket and acceptance criteria before writing code.
2. Identify dependencies, open questions, and security implications.
3. Implement the smallest complete change that satisfies the ticket.
4. Run the relevant local/manual test immediately.
5. Add or update automated tests when the ticket requires them.
6. Review migrations and API response fields before committing.
7. Ask: “Can I explain why I designed it this way?”
8. Only mark the ticket Done after acceptance criteria pass.

7. Current Checkpoint

Based on the current implementation session, Vault has completed the foundation work through the signup endpoint. The next planned ticket is VLT-112 — Implement Login Endpoint, followed by protected-current-user verification, tenant isolation demonstration, and automated Sprint 1 tests.

Next ticket: VLT-112 — Implement Login Endpoint